

git基本命令大全速查

文章目录

- [git add](#)
- [git blame](#)
- [git branch](#)
- [git checkout](#)
- [git cherry-pick](#)
- [git clone](#)
- [git commit](#)
- [git config](#)
- [git diff](#)
- [git fetch](#)
- [git init](#)
- [git log](#)
- [git ls-files](#)
- [git merge](#)
- [git mv](#)
- [git push](#)
- [git pull](#)
- [git reflog](#)
- [git remote](#)
- [git reset](#)
- [git restore](#)
- [git revert](#)
- [git rm](#)
- [git shortlog](#)
- [git show](#)
- [git status](#)
- [git stash](#)
- [git switch](#)
- [git tag](#)
- [git whatchanged](#)

git add

添加到暂存区

```
1 | git add . # 添加当前目录下的所有文件到暂存区
2 | git add dir/ # 添加指定目录下的所有文件到暂存区
3 | git add file1 file2 # 只添加指定的文件到暂存区
   AI写代码shell
```

相关

```
1 | git reset HEAD -- . # 撤销暂存区的所有改动，但不会影响工作目录
2 | git diff --cached # 显示暂存区与上一次提交（本地仓库）之间的差异
   AI写代码shell
```

git blame

可以追溯一个指定文件的历史修改记录

```
1 | git blame <文件名> # 显示一个文件的逐行历史修改记录，包括作者和修改时间
2 | git blame -L 69,82 <文件名> # 显示文件第69--82行最近一次的修改记录
3 | git blame --line-porcelain <文件名> # 显示文件修改历史，不包含文件名和行号
4 | git blame --show-email <文件名> # 显示文件的修改历史，只显示行号和作者
AI写代码shell
```

git branch

用于列出、创建或删除分支

```
1 | git branch # 列出本地所有分支
2 | git branch <分支名> # 创建一个新的分支
3 | git branch -d <分支名> # 删除指定的分支
4 | git branch -m <旧分支名> <新分支名> # 重命名一个分支
5 | git branch -r # 列出远程所有分支
6 | git branch -a # 列出本地和远程所有分支
7 | git branch -v # 列出本地所有分支并显示最后一次提交
8 | git branch -vv # 列出本地所有的分支并显示最后一次提交和对应的远程分支
AI写代码shell
```

git checkout

用于切换分支或恢复工作目录的文件，它是git最常用的命令之一，同时也是一个很危险的命令，因为这条命令会重写工作区。

```
1 | git checkout <分支名> # 切换到指定的分支
2 | git checkout -b <新分支名> # 创建并切换到一个新的分支，相当于 git branch <分支名> 再 git checkout <分支名>
3 | git checkout <提交ID> # 恢复到指定的提交
4 | git checkout <文件名> # 恢复指定文件到暂存区或最后一次提交状态
5 | git checkout -- <文件名> # 丢弃工作目录中对指定文件的修改
6 | git checkout <分支名> -- <文件名> # 从指定分支恢复文件到当前分支
7 | git checkout -b <分支名> origin/<分支名> # 克隆完远程分支后，默认只有master分支，此命令切换到远程分支
8 | git checkout <提交ID> -- <文件名> # 恢复某一文件。如果在暂存区有就从暂存区恢复，否则从版本库恢复。
AI写代码shell
```

git cherry-pick

用于将一个或多个提交的更改应用到当前分支

```
1 | git cherry-pick <提交ID> # 将指定的提交应用到当前分支
2 | git cherry-pick <提交ID1> <提交ID2> # 将多个指定的提交依次应用到当前分支
3 | git cherry-pick <提交ID1>..<提交ID2> # 将一段范围内的提交应用到当前分支
4 | git cherry-pick --continue # 解决冲突后继续应用提交
5 | git cherry-pick --abort # 取消当前的cherry-pick操作
6 | git cherry-pick --quit # 退出当前的cherry-pick操作但保留已经应用的提交
AI写代码shell
```

git clone

用于从远程仓库克隆一个完整的仓库到本地

```
1 | git clone <仓库地址> # 克隆远程仓库到本地
2 | git clone <仓库地址> <目录> # 克隆远程仓库到指定的本地目录
3 | git clone -b <分支名> <仓库地址> # 克隆指定分支的远程仓库
4 | git clone --depth <深度> <仓库地址> # 进行浅克隆，只获取指定深度的提交记录
5 | git clone --single-branch <仓库地址> # 只克隆指定分支，而不获取其他分支
6 | git clone --recurse-submodules <仓库地址> # 克隆远程仓库并初始化、更新所有子模块
AI写代码shell
```

git commit

用于提交暂存区的更改到本地仓库

```
1 git commit # 把暂存区中的文件提交到本地仓库, 调用文本编辑器输入提交信息
2 git commit -m "<提交信息>" # 提交暂存区的更改并添加提交信息
3 git commit -a -m "<提交信息>" # 跳过暂存区, 直接提交所有已修改的文件并添加提交信息
4 git commit --amend -m "<新提交信息>" # 修改上一次提交的信息或包括新的更改
5 git commit --amend --no-edit # 修改上一次提交, 不改变提交信息
6 git commit -v # 提交时显示所有差异
7 git commit --squash <提交ID> # 将当前更改与指定提交合并为一个提交
8 git commit --fixup <提交ID> # 创建一个修复指定提交的提交
```

AI写代码shell

从本地仓库撤销到暂存区: `git reset --soft HEAD^`

git config

用于配置Git的设置, 例如用户信息、编辑器等

```
1 git config --global user.name "<用户名>" # 设置全局用户名
2 git config --global user.email "<邮箱地址>" # 设置全局用户邮箱
3 git config --global core.editor "<编辑器>" # 设置全局默认编辑器
4 git config --list # 列出所有当前配置
5 git config <key> # 获取指定配置项的值
6 git config --global alias.<别名> "<命令>" # 创建全局别名
7 git config --system --list # 列出系统范围内的配置
```

AI写代码shell

git diff

查看差异

```
1 git diff # 显示工作目录中所有文件的尚未暂存的改动
2 git diff <文件名> # 显示指定文件的尚未暂存的改动
3 git diff --cached # 显示已暂存到索引但尚未提交的改动
4 git diff --stat # 以统计方式显示改动, 列出哪些文件被修改以及修改的行数
5 git diff HEAD # 显示工作目录和最新提交之间的所有改动
6 git diff HEAD^ # 显示工作目录和上一个提交之间的改动
7 git diff HEAD~5 # 显示工作目录和前五次提交之前的改动
8 git diff <提交ID> # 显示工作目录和指定提交之间的改动
9 git diff <分支名> # 显示当前分支与指定分支之间的差异
10 git diff <分支名称> <分支名称> # 显示两个指定分支之间的差异
11 git diff <分支名称>...<分支名称> # 显示两个分支的共同祖先以来的差异
12 git diff origin/dev dev # 显示远程分支origin/dev和本地分支dev之间的差异
```

AI写代码shell



git fetch

用于从远程仓库获取最新的更新, 但不会自动合并这些更新

```
1 git fetch # 从默认的远程仓库 (通常是 origin) 获取所有分支的更新
2 git fetch <远程仓库名> # 从指定的远程仓库获取更新
3 git fetch <远程仓库名> <分支名> # 从指定的远程仓库获取指定分支的更新
4 git fetch --all # 从所有配置的远程仓库获取更新
5 git fetch --prune # 获取更新时删除远程仓库中已删除的分支
6 git fetch origin # 从远程仓库 origin 获取更新
7 git fetch origin master # 从远程仓库 origin 获取 master 分支的更新
8 git fetch origin +refs/heads/*:refs/remotes/origin/* # 强制更新所有远程分支到本地
```

AI写代码shell

git init

用于初始化一个新的 Git 仓库

```
1 | git init # 在当前目录初始化一个新的 Git 仓库
2 | git init <目录名> # 在指定目录下初始化一个新的 Git 仓库
3 | git init --bare # 初始化一个裸仓库, 不包含工作目录, 通常用于远程仓库
4 | git init --template=<模板目录> # 使用指定的模板目录初始化一个新的 Git 仓库
5 | git init --separate-git-dir=<git目录> # 将 Git 仓库的元数据存储在与指定目录, 而不是默认的 .git 目录
AI写代码shell
```

git log

用于查看仓库的提交历史记录

```
1 | git log # 显示所有提交的历史记录
2 | git log -n 5 # 显示最近的 5 次提交
3 | git log --oneline # 简洁地显示提交历史, 每个提交一行
4 | git log --graph # 以图形方式显示提交历史
5 | git log --stat # 显示每次提交的文件修改统计
6 | git log --author="作者名" # 只显示指定作者的提交
7 | git log --since="2023-01-01" # 显示自指定日期以来的提交
8 | git log --until="2023-06-14" # 显示截至指定日期的提交
9 | git log --grep="关键字" # 只显示提交信息中包含指定关键字的提交
10 | git log <分支名> # 显示指定分支的提交历史
11 | git log develop -n 3 # 显示 develop 分支最近的 3 次提交
12 | git log origin/develop -n 3 # 显示远程仓库 origin 的 develop 分支最近的 3 次提交
13 | git log <提交ID> # 从指定提交开始显示提交历史
AI写代码shell
```



git ls-files

用于显示当前工作区中被 Git 跟踪的文件列表

```
1 | git ls-files # 显示当前工作区中所有被 Git 跟踪的文件
2 | git ls-files -o # 显示当前工作区中未被 Git 跟踪的文件
3 | git ls-files -i # 显示当前工作区中被忽略的文件
4 | git ls-files -m # 显示当前工作区中被修改但未提交的文件
5 | git ls-files -s # 显示文件的状态信息, 包括文件模式、对象名和文件名
6 | git ls-files --stage # 显示文件的索引信息
7 | git ls-files -d # 显示已被删除但尚未提交的文件
AI写代码shell
```

git merge

用于合并不同分支的内容

```
1 | git merge <分支名> # 将指定分支合并到当前分支
2 | git merge --no-ff <分支名> # 禁用快速合并, 确保创建一个新的合并提交
3 | git merge --squash <分支名> # 将指定分支的所有提交压缩为一个提交
4 | git merge --abort # 取消当前的合并过程并恢复到合并前的状态
5 | git merge --no-commit <分支名> # 合并指定分支但不自动提交
6 | git merge --strategy=recursive -X theirs <分支名> # 使用指定策略合并, 并在冲突时优先选择指定分支的内容
7 | git merge --strategy=recursive -X ours <分支名> # 使用指定策略合并, 并在冲突时优先选择当前分支的内容
AI写代码shell
```

git mv

用于移动或重命名被 Git 跟踪的文件或目录

```
1 git mv <旧文件路径> <新文件路径> # 将文件从旧路径移动到新路径
2 git mv <旧目录路径> <新目录路径> # 将目录从旧路径移动到新路径
3 git mv -f <旧文件路径> <新文件路径> # 强制移动文件, 即使目标路径已存在文件
4 git mv <旧文件路径1> <旧文件路径2> <新目录路径> # 将多个文件移动到新目录
AI写代码shell
```

git push

用于将本地分支的提交推送到远程仓库

```
1 git push <远程仓库名> <分支名> # 将指定分支推送到远程仓库
2 git push origin main # 将本地主分支推送到名为 origin 的远程仓库
3 git push --all # 推送所有本地分支到远程仓库
4 git push --tags # 推送所有本地标签到远程仓库
5 git push -u origin main # 推送本地主分支到远程仓库并设置跟踪
6 git push --force # 强制推送, 即使会覆盖远程仓库的历史记录
7 git push origin :<分支名> # 删除远程仓库中的指定分支
8 git push <远程主机名> <本地分支名>:<远程分支名> # 将本地分支推送到远程分支, 可以用于重命名远程分支
9 git push origin HEAD:<分支名> # 将当前分支推送到远程的新分支
10 git push --set-upstream origin <分支名> # 将本地分支推送到远程分支, 并设置本地分支与远程分支的跟踪关系
AI写代码shell
```



git pull

用于从远程仓库获取最新的更改并合并到当前分支。首先会执行 git fetch, 然后执行 git merge, 把获取的分支的 HEAD 合并到当前分支。

```
1 git pull <远程仓库名> <分支名> # 从指定的远程仓库和分支拉取更新并合并到当前分支
2 git pull origin main # 从名为 origin 的远程仓库的主分支拉取更新并合并到当前分支
3 git pull --rebase # 拉取更新并在本地提交之上重新应用 (rebase) 远程提交
4 git pull --ff-only # 仅执行快速合并 (fast-forward), 如果不能快速合并则终止
5 git pull --no-commit # 拉取更新但不自动提交合并结果
6 git pull --squash # 拉取更新并将所有更改压缩为一个提交
AI写代码shell
```

git reflog

用于查看本地仓库的引用日志, 记录了所有分支和 HEAD 的移动历史

git-reflog是用来恢复本地错误操作很重要的一个命令

```
1 git reflog # 显示所有引用的日志记录
2 git reflog show <分支名> # 显示指定分支的引用日志
3 git reflog show HEAD@{2} # 显示 HEAD 在两次前的状态
4 git reflog expire --expire=now --all # 清除所有引用日志
5 git reflog delete --rewrite <分支名>@{n} # 删除指定引用日志条目
AI写代码shell
```

git remote

用于管理远程仓库的链接

```
1 git remote # 列出所有远程仓库
2 git remote -v # 显示所有远程仓库的详细信息, 包括 fetch 和 push 的 URL
3 git remote add <远程仓库名> <仓库URL> # 添加一个新的远程仓库
4 git remote remove <远程仓库名> # 删除一个远程仓库
5 git remote rename <旧远程仓库名> <新远程仓库名> # 重命名一个远程仓库
6 git remote set-url <远程仓库名> <新URL> # 修改远程仓库的 URL 地址
AI写代码shell
```

git reset

用于重置当前分支的 HEAD 到指定状态，可以更改暂存区和工作目录

语法： `git reset [--soft | --mixed | --hard] [HEAD]`

```
1 | git reset --soft <提交ID> # 重置到指定提交，但保留暂存区和工作目录的更改
2 | git reset --soft HEAD^ # 本地仓库撤回回到暂存区，相当于调用 "git reset <commit ID>" 命令后又做了一次 "git add"
3 | git reset --mixed <提交ID> # 重置到指定提交，保留工作目录的更改，但清空暂存区（默认方式）
4 | git reset --hard <提交ID> # 重置到指定提交，清空暂存区和工作目录的所有更改
5 | git reset HEAD~1 # 重置到上一个提交，保留工作目录的更改，但清空暂存区
6 | git reset <文件名> # 从暂存区中移除指定文件的更改，但保留工作目录的更改，相当于撤销git add，可以指定文件也可以不指定
```

AI写代码shell

git restore

用于恢复工作目录或暂存区的文件

```
1 | git restore <文件名> # 恢复工作目录中的指定文件到最近一次提交的状态
2 | git restore --staged <文件名> # 从暂存区中移除指定文件的更改，但保留工作目录的更改
3 | git restore --source=<提交ID> <文件名> # 恢复指定提交中的文件到工作目录
4 | git restore --worktree <文件名> # 恢复工作目录中的指定文件（默认行为）
5 | git restore --staged --worktree <文件名> # 同时恢复暂存区和工作目录中的指定文件
```

AI写代码shell

git revert

生成一个新的提交来撤销某次提交，此次提交之前的所有提交都会被保留

```
1 | git revert <提交ID> # 创建一个新的提交，撤销指定的提交
2 | git revert -n <提交ID> # 创建一个新的提交，撤销指定的提交，但不自动提交更改
3 | git revert --no-edit <提交ID> # 创建一个新的提交，撤销指定的提交，并跳过提交信息编辑
4 | git revert HEAD # 撤销最近的一次提交
5 | git revert HEAD~2..HEAD # 撤销最近的两次提交
```

AI写代码shell

git rm

用于从暂存区和工作目录中删除文件

```
1 | git rm <文件名> # 从暂存区和工作目录中删除指定文件
2 | git rm --cached <文件名> # 仅从暂存区中删除指定文件，但保留工作目录中的文件
3 | git rm -r <目录名> # 递归删除指定目录及其内容
4 | git rm -f <文件名> # 强制删除已修改或已暂存的文件
5 | git rm -n <文件名> # 显示将要删除的文件，但不实际删除
```

AI写代码shell

git shortlog

用于生成提交历史的简短摘要，按作者分组

```
1 | git shortlog # 显示所有提交的简短摘要，按作者分组
2 | git shortlog -s # 只显示每个作者的提交数量
3 | git shortlog -n # 按提交数量排序作者
4 | git shortlog -e # 显示每个作者的电子邮件地址
5 | git shortlog v1.0..HEAD # 显示从标签v1.0到当前HEAD的提交摘要
```

AI写代码shell

git show

用于显示各种类型的对象（如提交、标签等）的详细信息


```
1 | git show <对象> # 显示指定对象的详细信息, 通常是提交
2 | git show # 显示最近一次提交的详细信息
3 | git show <提交ID> # 显示指定提交的详细信息
4 | git show <提交ID>:<文件名> # 显示指定提交中某个文件的内容
5 | git show --name-only <提交ID> # 显示指定提交的文件修改列表, 不包括具体内容
6 | git show --raw <提交ID> # 显示指定提交的详细信息, 包括文件修改的原始差异信息
7 | git show --stat <提交ID> # 显示指定提交的统计信息 (文件修改数量、插入和删除行数)
8 | git show --pretty=short <提交ID> # 以简短格式显示指定提交的日志信息
```

AI写代码shell

git status

用于显示工作目录和暂存区的状态。

```
1 | git status # 显示工作目录和暂存区的状态信息
2 | git status -s # 以简短格式显示状态信息
3 | git status --short # 同上, 简短格式显示状态信息
4 | git status -b # 显示当前分支信息以及与上游分支的差异
5 | git status --branch # 同上, 显示当前分支信息
6 | git status --ignored # 显示被忽略的文件
7 | git status --untracked-files=all # 显示所有未跟踪的文件
8 | git status --untracked-files=no # 不显示未跟踪的文件
9 | git status --untracked-files=normal # 默认选项, 显示未跟踪的文件
```

AI写代码shell

git stash

用于临时保存当前工作目录的修改, 以便在以后恢复

```
1 | git stash # 保存当前工作目录的修改到栈中
2 | git stash save "描述信息" # 保存当前工作目录的修改, 并添加描述信息
3 | git stash list # 显示所有保存的修改列表
4 | git stash apply # 恢复最近一次保存的修改, 但不删除该保存
5 | git stash apply stash@{2} # 恢复指定的保存 (例如第二个保存), 但不删除该保存
6 | git stash pop # 恢复最近一次保存的修改, 并删除该保存
7 | git stash drop # 删除最近一次保存的修改
8 | git stash drop stash@{2} # 删除指定的保存 (例如第二个保存)
9 | git stash clear # 清空所有保存的修改
10 | git stash show # 显示最近一次保存的修改内容
11 | git stash show stash@{2} # 显示指定保存的修改内容
12 | git stash branch <分支名> # 在新分支上应用最近一次保存的修改
```

AI写代码shell



git switch

用于切换分支或创建新分支

```
1 | git switch <分支名> # 切换到指定的分支
2 | git switch -c <新分支名> # 创建并切换到新分支
3 | git switch -C <新分支名> # 强制创建并切换到新分支, 如果分支已存在则覆盖
4 | git switch - # 切换到上一个分支
5 | git switch --detach <提交ID> # 切换到指定的提交, 不关联任何分支
```

AI写代码shell

git tag

用于给提交打标签, 通常用于标记版本

```
1 git tag # 列出所有标签
2 git tag <标签名> # 给当前最新提交打一个标签
3 git tag -a <标签名> -m "标签信息" # 创建带有信息的注释标签
4 git tag -d <标签名> # 删除指定的标签
5 git tag -l "v1.0.*" # 列出符合特定模式的标签, 例如所有以v1.0.开头的标签
6 git tag <标签名> <提交ID> # 给指定的提交打标签
7 git show <标签名> # 显示指定标签的详细信息
8 git push origin <标签名> # 推送指定标签到远程仓库
9 git push origin --tags # 推送所有标签到远程仓库
10 git push origin :refs/tags/<标签名> # 删除远程仓库中的指定标签
11 git checkout <标签名称> # 切换到指定的标签
12 git show <标签名称> # 查看标签的信息
AI写代码shell
```



git whatchanged

用于显示提交历史及其引入的文件变化

```
1 git whatchanged # 显示提交历史及其引入的文件变化
2 git whatchanged -p # 以补丁格式显示提交历史及其引入的文件变化
3 git whatchanged --since="2 weeks ago" # 显示两周以来的提交历史及其引入的文件变化
4 git whatchanged --until="2023-01-01" # 显示指定日期之前的提交历史及其引入的文件变化
5 git whatchanged -n 5 # 显示最近的5次提交历史及其引入的文件变化
6 git whatchanged <路径> # 显示指定路径的提交历史及其引入的文件变化
AI写代码shell
```